

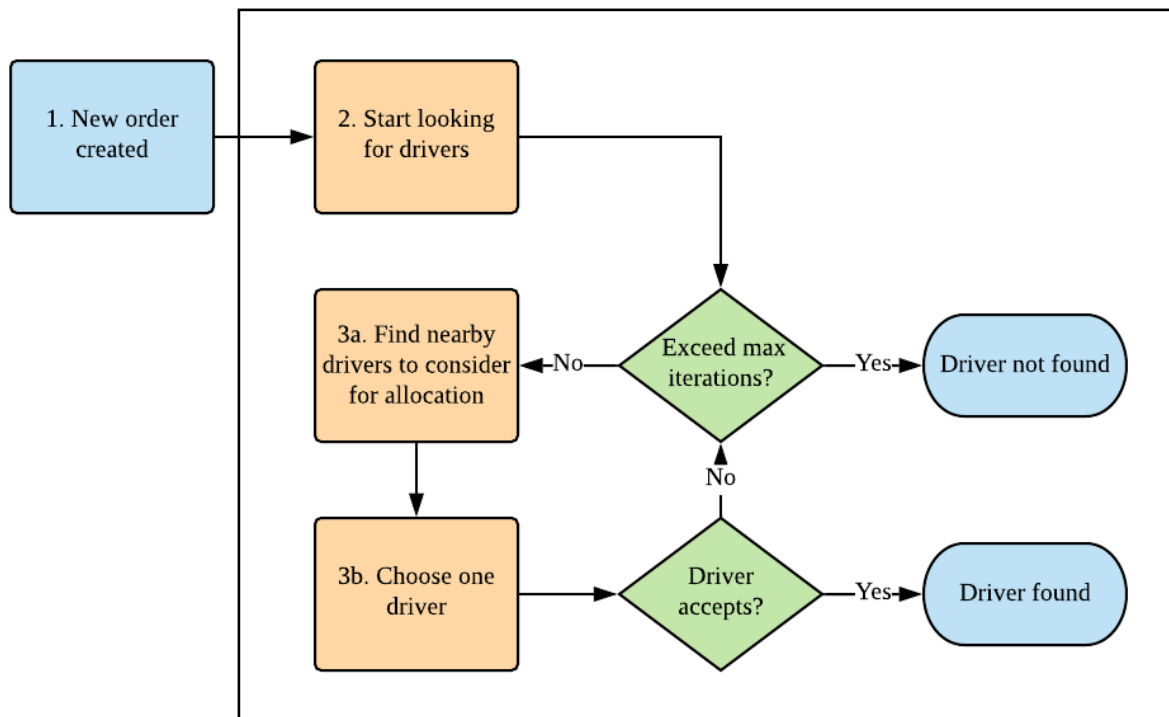


Congratulations on reaching the next stage of GoTo's Data Science recruitment process! The take-home assignment aims to provide you with a realistic taste of Data Science work at GoTo. It's inspired by one of the first problems the DS team tackled - driver allocation. The allocation flow has been simplified and synthetic data generated for this assignment but the key aspects of the business problem are captured.

Business Context

The driver allocation problem at GoTo is central to the business. Data scientists are responsible for determining the best driver to be allocated millions of times every day. We tap on huge amounts of data to optimize our decisions while taking into account dynamic business objectives that balance the needs of our drivers and customers. Your assignment is to optimize allocation for GoRide, our two-wheel transportation service. The allocation flow is as follows:

Allocation System



1. A customer enters their trip details and makes a booking request. The order management system creates an order and requests our allocation system to find a driver
2. The allocation system creates an experiment key for A/B testing and begins an iterative process to find a driver
3. While the iteration count is below some threshold:
 - a. we find available nearby drivers
 - b. we choose a driver and send a booking notification (in our nomenclature the driver becomes a 'participant' in the allocation flow at this point)
 - i. the driver accepts → we notify the customer (exit loop)
 - ii. the driver rejects or ignores → next iteration
4. If no driver has accepted → notify the customer and end flow
5. Else, the driver is dispatched to pick up the customer
 - a. before pick up, the driver may cancel
6. The customer may cancel their booking at any point, and this becomes more likely the longer they wait

We are currently focusing our efforts on improving step 3b - choosing the right driver, with the business objective of **increasing the number of accepted orders**.

Data Source

Event logs from our production systems will be made available to you via CSV files:

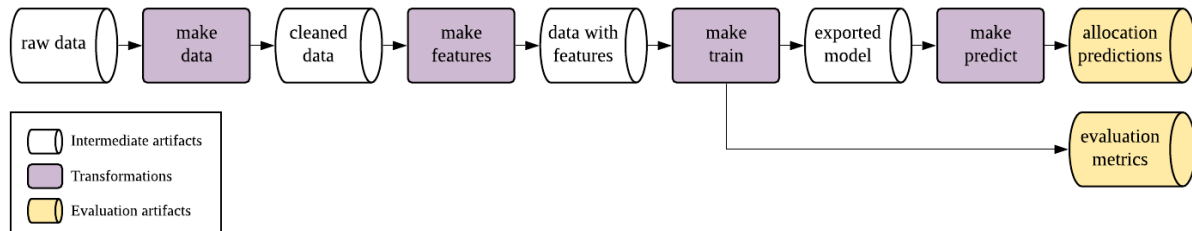
- **booking_log.csv** records order-level events: created, driver found/not found, canceled, completed
- **participant_log.csv** records events at the participant (driver) level: created, accepted, rejected, ignored

Note that the data may not be perfectly clean. There may be some missing events (e.g. if the driver does not record the pickup) or duplicate messages stored in our data warehouse.

Instructions

As a data scientist at GoTo, you will be expected to contribute to high impact projects with little supervision, while upholding good machine learning engineering principles. Candidates should adopt a similar mindset when attempting this assignment.

The objective of this assignment is to evaluate your ability to contribute to an existing model training pipeline. You are given a model training pipeline that predicts the best driver for each order, but you will need to fix several issues in the code to get it running. Think of this as a set of koans; run the pipeline, encounter an error or analyze the results, fix the problem, repeat.



There are three tasks to be completed:

1. **Fix the pipeline and run it end-to-end.** It should produce `submission/results.csv` and `submission/metrics.json`.
2. **Make one change to improve model performance.** This can take the form of anything including hyperparameter tuning, feature engineering, or an entirely new model.
3. **Summarize your findings in a one-page document.** This allows us to understand *how* you are thinking about the problem, in addition to *what* you did.

Submission Requirements

1. A .zip archive of your code. Ensure you are able to run `make run` with no issues.
2. A brief one-page writeup of your solution saved as PDF. We are interested in how you reason through the problem and how you arrived at your solutions.
3. Include git metadata (a .git folder), which should include readable git commit messages.
4. A `submission/metrics.json` file containing evaluation metrics for your model.
5. A `submission/results.csv` file containing the selected drivers with this schema:

order_id	driver_id
100032007	987454392
100167816	790186080
:	:

Windows OS Setup

Candidates using a Windows operating system may be unable to run some of the Make targets, which were designed to be run on unix-based operating systems. Affected candidates are advised to complete the assignment using a Windows Subsystem for Linux, by following these steps:

1. [Install Windows Subsystem for Linux](#) (WSL1 or WSL2)
2. Navigate to the assignment folder in your windows filesystem which is stored in `/mnt/c/`
3. Edit the code in the IDE of your choice
4. Run the necessary make targets via the WSL terminal

Alternatively, candidates using Visual Studio Code may opt to install the [WSL extension](#) instead.

Tips

- **This is not a Kaggle competition.** We care more about your approach than your score. Communicate your observations, thought processes, and conclusions.
- **Read the code expecting to find bugs.** Some errors will be explicit, others will be silent (as is often the case in machine learning). Spotting them will be key. In particular, pay attention to your datasets - not everything is what it seems.
- **Default to Occam's razor.** We won't be particularly impressed by complex deep learning solutions for the sake of it, especially if simpler models are a viable alternative.
- **Practice good engineering hygiene.** Approach this assignment as though you are contributing to a production codebase. Do not write hacky code. Ensure that your work is reproducible and your code, robust.
- **Be curious.** You can read more about how we tackled driver allocation [here](#).
- **Do not reinvent the wheel.** Use third-party libraries appropriately.

Terms & Conditions

1. Don't make the problem statement, dataset or your solution publicly available by, for example, posting this problem on a blog, forum, or public code repositories.
2. Don't include any executable files, class files, .pyc files, jars or other libraries or output from the build process in your submission.